ODOO DEVELOPER PROGRAM

Web Development and Integrations

Odoo 17 Development Tutorials

Ahmed Muhumed

July 26, 2026

Outline

Introduction

What Will We Learn?

- Controllers, HTTP routes, and JSON controllers
- REST-style APIs, XML-RPC, and JSON-RPC
- External API integration patterns
- File upload and download
- OWL framework, JavaScript, components, services, hooks
- Assets and custom widgets

Backend Web vs Frontend Web in Odoo

- **Backend web**: Python controllers expose HTTP/JSON endpoints.
- **External APIs**: XML-RPC / JSON-RPC let outside systems talk to Odoo.
- **Frontend web**: OWL + JavaScript build interactive UI in the browser.
- Together they power websites, portals, custom backends, and integrations.

Chapter Map

1. Controllers & routes
2. JSON / REST endpoints
3. RPC protocols and external integration
4. Files in and out of Odoo
5. OWL frontend architecture

Controllers

What Is a Controller?

- A **controller** is a Python class that handles HTTP requests.
- In Odoo it inherits from `http.Controller`.
- Controllers define routes that return HTML, JSON, files, or redirects.

Method Syntax

```python
from odoo import http
from odoo.http import request

class StudentController(http.Controller):
    pass
```

Controller File Placement

```
school_manager/
  controllers/
  __init__.py
  student_portal.py
  __init__.py
```

```python
# school_manager/__init__.py
from . import controllers

# school_manager/controllers/__init__.py
from . import student_portal
```

- Controllers must be imported or Odoo will not register the routes.

request Object

- Inside controllers, `request` is the current HTTP request.

```
request.env           # environment for current user/db
request.httprequest   # underlying Werkzeug request
request.session       # session data
request.params        # combined request parameters
```

- Most business work uses `request.env['model.name']`.

Simple HTML Controller

```python
class StudentController(http.Controller):

    @http.route('/school/students', type='http', auth='public', website=True)
    def student_list(self, **kwargs):
        students = request.env['student.student'].sudo().search([])
        return request.render('school_manager.student_list_page', {
            'students': students,
        })
```

- `request.render()` renders a QWeb website template.

Controller Best Practices

- Keep controllers thin; put business logic in models.
- Choose `auth` carefully (`public`, `user`, `none`).
- Validate input before writing to the database.
- Use `sudo()` only when the route intentionally needs elevated access.

HTTP Routes

What Is @http.route?

- `@http.route` registers a URL endpoint on a controller method.

Method Syntax

```
@http.route('/school/hello', type='http', auth='public', methods=['GET'], csrf=
    False)
def hello(self, **kwargs):
  return "Hello from Odoo"
```

- This is the foundation of Odoo web development.

Important Route Options

- **type**: `http` or `json`
- **auth**: `public`, `user`, `none`
- **methods**: `['GET']`, `['POST']`, ...
- **website**: enable website context/rendering
- **csrf**: CSRF protection for form posts
- **cors**: cross-origin configuration when needed

Auth Modes Explained

- `auth='public'`: website visitors can access; env may be public user
- `auth='user'`: logged-in users only
- `auth='none'`: no DB/session user setup initially (special cases)

```python
@http.route('/my/students', type='http', auth='user', website=True)
def my_students(self, **kwargs):
    students = request.env['student.student'].search([
        ('user_id', '=', request.env.user.id),
    ])
    return request.render('school_manager.my_students', {'students': students})
```

Path Parameters

```python
@http.route('/school/student/<int:student_id>', type='http', auth='public',
    website=True)
def student_detail(self, student_id, **kwargs):
  student = request.env['student.student'].sudo().browse(student_id)
  if not student.exists():
    return request.not_found()
  return request.render('school_manager.student_detail', {
    'student': student,
  })
```

- Converters include `int`, `string`, and more.

GET vs POST

```python
@http.route('/school/search', type='http', auth='public', methods=['GET'], website
    =True)
def search_students(self, q='', **kwargs):
  students = request.env['student.student'].sudo().search([
    ('name', 'ilike', q),
  ])
  return request.render('school_manager.student_search', {
    'students': students,
    'q': q,
  })
```

- Use GET for reads/search pages.
- Use POST for create/update actions and forms.

HTTP Route Best Practices

- Keep URLs clear and namespaced (`/school/...`).
- Restrict methods explicitly.
- Validate and sanitize query/form parameters.
- Return proper errors (`not_found`, `forbidden`) when needed.

JSON Controllers

What Is a JSON Controller?

- A JSON controller route uses `type='json'`.
- It receives JSON-RPC style calls and returns Python data (dict/list).
- Ideal for AJAX / OWL service calls inside Odoo web client.

Method Syntax

```
@http.route('/school/student/info', type='json', auth='user')
def student_info(self, student_id):
    student = request.env['student.student'].browse(student_id)
    return {
        'id': student.id,
        'name': student.name,
        'email': student.email,
    }
```

Calling a JSON Route from JS

```
/** @odoo-module **/
import { jsonrpc } from "@web/core/network/rpc_service";

const result = await jsonrpc("/school/student/info", {
  student_id: 15,
});
console.log(result.name);
```

- Frontend sends arguments; backend returns JSON-serializable data.

JSON vs HTTP Routes

- `type='http'`: returns HTML/text/files/redirects
- `type='json'`: returns JSON data for programmatic clients

```
# HTTP
return request.render(...)
return request.make_response(...)

# JSON
return {'ok': True, 'count': 3}
```

Validation and Errors

```python
@http.route('/school/student/update_email', type='json', auth='user')
def update_email(self, student_id, email):
    student = request.env['student.student'].browse(student_id)
    if not student.exists():
        return {'error': 'Student not found'}
    if not email or '@' not in email:
        return {'error': 'Invalid email'}
    student.write({'email': email})
    return {'ok': True}
```

JSON Controller Best Practices

- Return simple dict/list structures.
- Keep authentication strict (`auth='user'` by default).
- Do not leak internal exceptions to clients.
- Prefer model methods for reusable business logic.

REST API

REST Style in Odoo Controllers

- Odoo does not force a pure REST framework, but you can design REST-like HTTP APIs.
- REST ideas:
  - resources as URLs
  - HTTP verbs for actions (`GET`, `POST`, `PUT`, `DELETE`)
  - JSON request/response bodies
- Useful for mobile apps and external backends.

REST-like Endpoints Example

```python
class StudentAPI(http.Controller):

  @http.route('/api/v1/students', type='http', auth='user',
        methods=['GET'], csrf=False)
  def list_students(self, **kwargs):
    students = request.env['student.student'].search_read(
      [], ['name', 'email']
    )
    return request.make_json_response(students)

  @http.route('/api/v1/students', type='http', auth='user',
        methods=['POST'], csrf=False)
  def create_student(self, **kwargs):
    data = request.get_json_data()
    student = request.env['student.student'].create(data)
    return request.make_json_response({'id': student.id}, status=201)
```

REST Design Tips

```
GET     /api/v1/students
GET     /api/v1/students/<id>
POST    /api/v1/students
PUT     /api/v1/students/<id>
DELETE  /api/v1/students/<id>
```

- Keep endpoints resource-oriented.
- Use proper status codes when building HTTP APIs.
- Protect with login, API keys, or custom auth as needed.

MAKE_JSON_RESPONSE

```python
return request.make_json_response(
  {'ok': True, 'student_id': student.id},
  status=200,
)
```

- Convenient helper for HTTP JSON APIs in modern Odoo.
- Alternative to pure `type='json'` when you want classical REST verbs.

REST Best Practices

- Version your API (`/api/v1/...`) when exposing publicly.
- Validate payloads strictly.
- Never expose elevated APIs without authentication and authorization.
- Document endpoints for integrators.

XML-RPC

What Is XML-RPC in Odoo?

- **XML-RPC** is a classic remote procedure protocol supported by Odoo.
- External systems call Odoo methods over HTTP using XML payloads.
- Common endpoints:
  - `/xmlrpc/2/common`
  - `/xmlrpc/2/object`

Use Case

Legacy ERP connectors, scripts, and external apps.

Authentication Flow

```python
import xmlrpc.client

url = "http://localhost:8069"
db = "school_db"
username = "admin"
password = "admin"

common = xmlrpc.client.ServerProxy(f"{url}/xmlrpc/2/common")
uid = common.authenticate(db, username, password, {})
print(uid)  # user id if login succeeds
```

Calling ORM Methods via XML-RPC

```
models = xmlrpc.client.ServerProxy(f"{url}/xmlrpc/2/object")

student_ids = models.execute_kw(
  db, uid, password,
  'student.student', 'search',
  [[('active', '=', True)]],
)

students = models.execute_kw(
  db, uid, password,
  'student.student', 'read',
  [student_ids, ['name', 'email']],
)
```

Create / Write / Unlink

```
new_id = models.execute_kw(
    db, uid, password,
    'student.student', 'create',
    [{'name': 'Ahmed Muhumed', 'email': 'ahmed@example.com'}],
)

models.execute_kw(
    db, uid, password,
    'student.student', 'write',
    [[new_id], {'age': 22}],
)
```

- Same ORM method names, called remotely.
- Access rights and record rules still apply to the authenticated user.

XML-RPC Notes

- Stable and widely supported.
- Verbose XML payloads compared to JSON-RPC.
- Great for simple external scripts.
- Prefer HTTPS in production.

JSON-RPC

What Is JSON-RPC in Odoo?

- **JSON-RPC** is Odoo's JSON-based remote call protocol.
- Endpoint: `/jsonrpc`
- Same idea as XML-RPC, but with JSON payloads.

Why Use It?

Easier for JavaScript / modern web and mobile clients.

Authenticate with JSON-RPC

```python
import requests

url = "http://localhost:8069/jsonrpc"
payload = {
  "jsonrpc": "2.0",
  "method": "call",
  "params": {
    "service": "common",
    "method": "authenticate",
    "args": ["school_db", "admin", "admin", {}],
  },
  "id": 1,
}
uid = requests.post(url, json=payload).json()["result"]
```

Search Read via JSON-RPC

```
payload = {
  "jsonrpc": "2.0",
  "method": "call",
  "params": {
    "service": "object",
    "method": "execute_kw",
    "args": [
      "school_db", uid, "admin",
      "student.student", "search_read",
      [[["active", "=", True]]],
      {"fields": ["name", "email"], "limit": 10},
    ],
  },
  "id": 2,
}
result = requests.post(url, json=payload).json()["result"]
```

XML-RPC vs JSON-RPC

- Both call the same Odoo ORM layer remotely.
- XML-RPC: XML payloads, Python stdlib support.
- JSON-RPC: JSON payloads, natural for JS/web stacks.
- For many new integrations, JSON-RPC is more convenient.

JSON-RPC Best Practices

- Store credentials securely; never hard-code in frontend public code.
- Use a dedicated integration user with least privilege.
- Handle `error` responses from JSON-RPC carefully.
- Prefer HTTPS and strong passwords/API policies.

External API Integration

Calling External APIs from Odoo

- Controllers/RPC expose Odoo to the outside world.
- External API integration is the reverse: Odoo calls another system.
- Examples:
  - payment gateways
  - SMS providers
  - national ID verification
  - shipping / accounting services

Basic Outbound HTTP Call

```python
import requests
from odoo import models, api
from odoo.exceptions import UserError

class Student(models.Model):
    _inherit = 'student.student'

    def action_sync_external(self):
        self.ensure_one()
        url = self.env['ir.config_parameter'].sudo().get_param(
            'school_manager.external_api_url'
        )
        response = requests.post(url, json={
            'name': self.name,
            'email': self.email,
        }, timeout=20)
        if response.status_code >= 400:
            raise UserError("External API sync failed.")
        self.external_ref = response.json().get('id')
```

Integration Design Pattern

1. Store API URL/keys in `ir.config_parameter` or settings
2. Isolate call logic in a model method / service class
3. Handle timeouts, retries, and errors
4. Log request references for debugging
5. Optionally queue calls with cron for reliability

Safe Credential Handling

```
ICP = self.env['ir.config_parameter'].sudo()
api_key = ICP.get_param('school_manager.external_api_key')
headers = {'Authorization': f'Bearer {api_key}'}
```

- Do not commit secrets into public repositories.
- Restrict settings access to admins.
- Prefer environment-specific parameters.

External Integration Best Practices

- Keep external IO out of tight UI loops when possible.
- Use clear UserError messages for business failures.
- Write tests with mocked HTTP responses.
- Document mapping between Odoo fields and external payloads.

File Upload & Download

Files in Odoo Web

- Upload: browser sends files to a controller or into `ir.attachment`.
- Download: controller returns a file response.
- Attachments are the standard storage model for many files.

Upload via Controller

```python
import base64

@http.route('/school/student/upload', type='http', auth='user',
        methods=['POST'], website=True)
def upload_student_file(self, student_id, ufile, **post):
  student = request.env['student.student'].browse(int(student_id))
  request.env['ir.attachment'].create({
    'name': ufile.filename,
    'res_model': 'student.student',
    'res_id': student.id,
    'type': 'binary',
    'datas': base64.b64encode(ufile.read()),
    'mimetype': ufile.content_type,
  })
  return request.redirect(f'/school/student/{student.id}')
```

Download a File

```python
@http.route('/school/attachment/<int:attachment_id>/download',
        type='http', auth='user')
def download_attachment(self, attachment_id, **kwargs):
  attachment = request.env['ir.attachment'].browse(attachment_id)
  if not attachment.exists():
    return request.not_found()
  return request.make_response(
    base64.b64decode(attachment.datas),
    headers=[
      ('Content-Type', attachment.mimetype or 'application/octet-stream'),
      ('Content-Disposition',
       f'attachment; filename="{attachment.name}"'),
    ],
  )
```

Binary Fields vs Attachments

- **Binary field**: file stored on the record field

```
photo = fields.Binary(string='Photo', attachment=True)
```

- **ir.attachment**: separate attachment records linked by model/res_id
- Use attachments for multiple documents per record.

File Handling Best Practices

- Validate file type/size before accepting uploads.
- Check record access before download.
- Prefer `attachment=True` for large binary fields.
- Never trust original filenames blindly in headers/paths.

OWL Framework

What Is OWL?

- **OWL** (Odoo Web Library) is Odoo's modern frontend framework.
- Component-based, reactive, and inspired by ideas from React-like systems.
- Used heavily in Odoo 17 web client, OWL apps, and custom UI.

Core Idea

UI is built from components that own state, templates, and lifecycle.

Why OWL in Odoo?

- Replaces older widget-heavy JS patterns for many new UIs.
- Better composition through components.
- Services provide shared browser-side features (RPC, notification, etc.).
- Hooks and lifecycle methods make state easier to manage.

Minimal OWL Component

```
/** @odoo-module **/
import { Component } from "@odoo/owl";

export class HelloStudent extends Component {
  static template = "school_manager.HelloStudent";
  static props = {
    name: { type: String },
  };
}
```

```
<t t-name="school_manager.HelloStudent">
  <div class="o_hello_student">
  Hello <t t-esc="props.name"/>
  </div>
</t>
```

OWL Building Blocks

- **Component**: reusable UI unit
- **Template**: XML/QWeb-like structure for rendering
- **Props**: inputs from parent
- **State**: reactive internal data
- **Services**: shared global features
- **Hooks**: lifecycle and utility helpers

OWL Best Practices

- Keep components small and focused.
- Pass data through props; avoid hidden globals.
- Put shared logic in services when many components need it.
- Register assets correctly so OWL modules load.

JavaScript

- Odoo frontend code is organized as ES modules.
- Files usually start with:

```
1 /** @odoo-module **/
```

- Modules import from Odoo packages such as:
  - `@odoo/owl`
  - `@web/core/...`
  - `@web/views/...`

Module Import / Export

```
/** @odoo-module **/
import { Component, useState } from "@odoo/owl";
import { registry } from "@web/core/registry";

export class StudentCounter extends Component {
  static template = "school_manager.StudentCounter";
  setup() {
    this.state = useState({ value: 0 });
  }
  increment() {
    this.state.value++;
  }
}
```

Registering Frontend Pieces

```
registry.category("actions").add("school_student_dashboard", StudentDashboard);
registry.category("fields").add("student_badge", studentBadgeField);
```

- Registries are how Odoo discovers actions, fields, views, services, and more.
- Custom JS usually extends an existing registry category.

Debugging Tips

- Use browser devtools extensively.
- Check Assets for load/order issues after upgrades.
- Prefer small modules over giant JS files.
- Keep naming consistent with module technical names.

JavaScript Best Practices

- Always include `/** @odoo-module **/`.
- Prefer official `@web` / `@odoo/owl` imports.
- Avoid editing core Odoo JS; extend via your module assets.
- Write intentional UI logic, not copies of backend business rules.

Components

What Is an OWL Component?

- A component is a class + template (+ optional props/state).
- Parents compose children to build complex screens.

Shape

```
export class StudentCard extends Component {
  static template = "school_manager.StudentCard";
  static props = ["student"];
  static components = {};
}
```

State with useState

```
import { Component, useState } from "@odoo/owl";

export class StudentFilter extends Component {
  static template = "school_manager.StudentFilter";
  setup() {
    this.state = useState({
      query: "",
      onlyActive: true,
    });
  }
  onQueryInput(ev) {
    this.state.query = ev.target.value;
  }
}
```

- Changing reactive state re-renders the component.

Props and Child Components

```
export class StudentList extends Component {
  static template = "school_manager.StudentList";
  static components = { StudentCard };
  static props = {
    students: { type: Array },
  };
}
```

```
<t t-name="school_manager.StudentList">
  <t t-foreach="props.students" t-as="student" t-key="student.id">
  <StudentCard student="student"/>
  </t>
</t>
```

Events / Callbacks

```
export class StudentCard extends Component {
  static props = {
    student: { type: Object },
    onSelect: { type: Function, optional: true },
  };
  select() {
    this.props.onSelect?.(this.props.student);
  }
}
```

- Prefer callbacks/props over tightly coupled parent lookups.

Component Best Practices

- One component = one UI responsibility.
- Declare `static props` clearly.
- Use `t-key` in loops.
- Keep templates close to the component's module assets.

Services

What Are Services?

- **Services** are shared frontend features available to many components.
- Examples from Odoo web:
  - `rpc` / ORM calls
  - `notification`
  - `action`
  - `dialog`
  - `user`
- Access them with `useService(...)`.

Using Built-in Services

```
import { Component, useState } from "@odoo/owl";
import { useService } from "@web/core/utils/hooks";

export class StudentRefresh extends Component {
  static template = "school_manager.StudentRefresh";
  setup() {
    this.orm = useService("orm");
    this.notification = useService("notification");
    this.state = useState({ students: [] });
  }
  async load() {
    this.state.students = await this.orm.searchRead(
      "student.student",
      [["active", "=", true]],
      ["name", "email"]
    );
    this.notification.add("Students loaded", { type: "success" });
  }
}
```

Custom Service Skeleton

```
/** @odoo-module **/
import { registry } from "@web/core/registry";

export const studentService = {
  dependencies: ["orm", "notification"],
  start(env, { orm, notification }) {
    return {
      async countActive() {
        const n = await orm.searchCount("student.student", [[["active", "=", true]]]);
        notification.add(`Active students: ${n}`);
        return n;
      },
    };
  },
};

registry.category("services").add("student_service", studentService);
```

Using a Custom Service

```
setup() {
  this.studentService = useService("student_service");
}
async onClick() {
  await this.studentService.countActive();
}
```

- Custom services are ideal for reusable frontend business helpers.

Service Best Practices

- Use services for shared logic, not one-off component details.
- Declare dependencies explicitly.
- Keep services side-effect aware and easy to test.
- Prefer ORM/RPC services over handmade fetch calls when talking to Odoo.

Hooks

What Are Hooks?

- Hooks are functions used inside `setup()` to add behavior to components.
- They help with:
  - reactive state
  - services
  - lifecycle
  - DOM refs
  - component environment utilities

Common OWL / Odoo Hooks

```
import { useState, onWillStart, onMounted } from "@odoo/owl";
import { useService } from "@web/core/utils/hooks";

setup() {
  this.orm = useService("orm");
  this.state = useState({ students: [] });

  onWillStart(async () => {
    this.state.students = await this.orm.searchRead(
      "student.student", [], ["name"]
    );
  });

  onMounted(() => {
    console.log("Student component mounted");
  });
}
```

Lifecycle Hooks

- `onWillStart`: before first render (async loading fits here)
- `onMounted`: after first DOM mount
- `onWillUpdateProps`: before props change is applied
- `onPatched`: after re-render/DOM patch
- `onWillUnmount`: cleanup

Custom Hook Example

```
import { useState, onWillStart } from "@odoo/owl";
import { useService } from "@web/core/utils/hooks";

export function useActiveStudents() {
  const orm = useService("orm");
  const state = useState({ records: [], ready: false });
  onWillStart(async () => {
    state.records = await orm.searchRead(
      "student.student",
      [["active", "=", true]],
      ["name", "email"]
    );
    state.ready = true;
  });
  return state;
}
```

Hooks Best Practices

- Call hooks only inside `setup()` (or custom hooks).
- Keep `onWillStart` loading logic clear and failure-tolerant.
- Extract reusable hook logic when multiple components need it.
- Clean up listeners/timers in unmount hooks.

Assets and Custom Widgets

What Are Assets?

- Assets are JS, CSS, SCSS, and XML frontend files loaded by Odoo.
- In Odoo 17 they are usually declared in `__manifest__.py`.

```
'assets': {
    'web.assets_backend': [
        'school_manager/static/src/**/*',
    ],
    'web.assets_frontend': [
        'school_manager/static/src/website/**/*',
    ],
},
```

Typical Frontend Folder Layout

```
school_manager/
  static/
  src/
    components/
    student_card.js
    student_card.xml
    student_card.scss
    services/
    student_service.js
    js/
    student_badge_field.js
```

Custom Field Widget (OWL)

```
/** @odoo-module **/
import { Component } from "@odoo/owl";
import { registry } from "@web/core/registry";
import { standardFieldProps } from "@web/views/fields/standard_field_props";

class StudentBadgeField extends Component {
  static template = "school_manager.StudentBadgeField";
  static props = { ...standardFieldProps };
  get label() {
    return this.props.record.data[this.props.name] || "N/A";
  }
}

registry.category("fields").add("student_badge", {
  component: StudentBadgeField,
});
```

Using the Widget in a View

```
<field name="code" widget="student_badge"/>
```

```
<t t-name="school_manager.StudentBadgeField">
  <span class="badge text-bg-primary" t-esc="label"/>
</t>
```

- The `widget=` name must match the registry key.

Custom Client Action

```
registry.category("actions").add("student_dashboard", StudentDashboard);
```

```
<record id="action_student_dashboard" model="ir.actions.client">
  <field name="name">Student Dashboard</field>
  <field name="tag">student_dashboard</field>
</record>
```

- Client actions open custom OWL screens inside the backend.

Assets & Widget Best Practices

- Put frontend files under `static/src`.
- Declare assets in the manifest; upgrade module after changes.
- Keep JS/XML/SCSS for one component together.
- Use clear registry names namespaced by your app.
- Test widgets in list/form views with empty and long values.

Web Development Summary

- Controllers + routes expose HTTP/JSON endpoints.
- REST/XML-RPC/JSON-RPC connect Odoo with external systems.
- File routes and attachments handle uploads/downloads.
- OWL components, services, hooks, and assets build modern UI.
- Custom widgets and client actions extend the backend experience.

Practical Checklist

- Need a webpage/endpoint? → Controller + `@http.route`
- Need AJAX data? → JSON controller / ORM service
- Need external system access? → XML-RPC / JSON-RPC / REST
- Need custom UI? → OWL component + assets + registry
- Need special field display? → custom field widget